



Running Breadboard Experiments on Prolific

Authored by Karen Gao
Updated by Eric Feltham (eric.feltham@aya.yale.edu)
Last updated: October 2026

Contents

1 Introduction 2
2 Setting Up Breadboard 2
3 Setting Up Prolific 5
3.1 Completion Paths Set Up 7
4 Pre-Launch Checklist 7
5 Running the Experiment 9
5.1 Session Timeline 9
5.2 Terminal Logs 9
5.3 Breadboard Console 9
5.4 Matching Players to Prolific Participants 10
5.5 Prolific 10
6 Running Group Studies 11
7 Troubleshooting 12
8 Testing Experiments on Breadboard 12
8.1 Testing with AI 12
8.2 Testing with Breadboard's Testing Dialog 13
9 Data Handling 13
10 Conclusion 14
Appendix A: Revolution Game Completion Codes 15
Appendix B: Revolution Game Bonus Script 16

1 Introduction

This guide explains how to set up and run experiments you create on Breadboard through Prolific, a platform that recruits users online to participate in research studies. Most steps apply to any study. Examples, server names and screenshots come from a revolution game study, so substitute your own study's values. Two parts are specific to that game and are in appendices: its completion codes (Appendix A) and its bonus script (Appendix B).

Prolific's interface changes periodically. Screenshots of the Prolific site show an earlier layout, so use them to find the right field and rely on the field names in the text.

Before you start, you need:

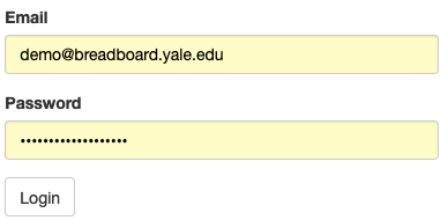
- Breadboard login details for your study's server, and SSH access if you want to watch the logs. Ask your Breadboard administrator for these.
- A Prolific account with a project for the study, and enough funds in the wallet for rewards, bonuses and Prolific's platform fee.
- A study that has been tested on Breadboard (see *Testing Experiments on Breadboard*).

Terms. A **player** is a participant as Breadboard sees them (a vertex in the graph, with a player ID). A **participant** is the same person as Prolific sees them (with a participant ID and a submission ID). The guide uses each word on the side where it applies, and the Prolific URL parameters link the two.

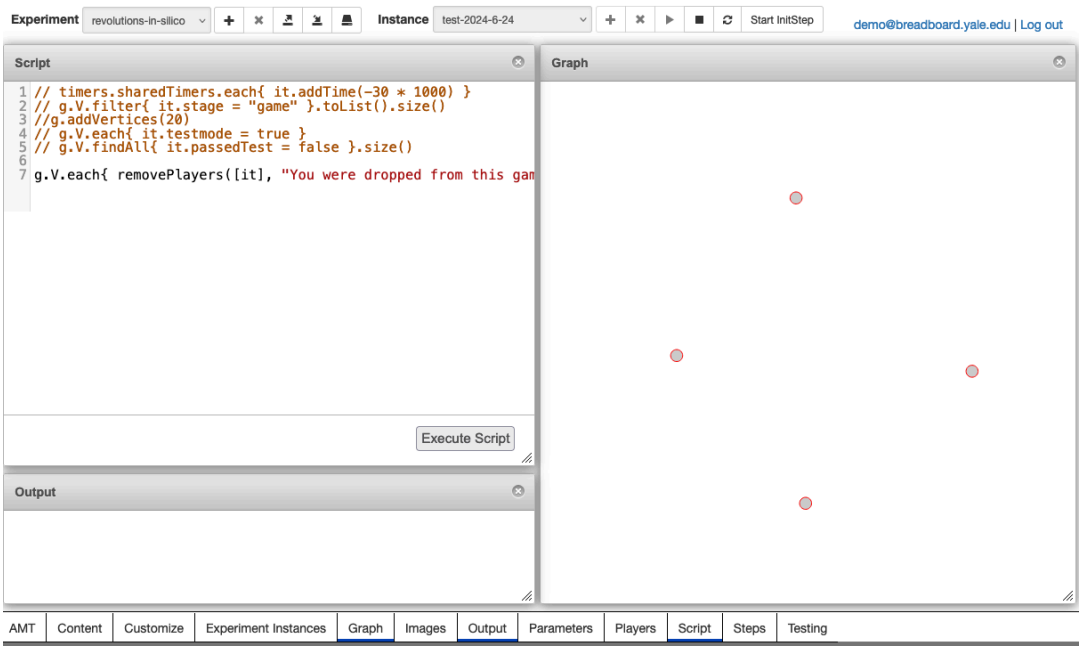
2 Setting Up Breadboard

1. Go to the URL of your Breadboard game and log in with your credentials.
   - URL (may look something like `https://[BREADBOARD_GAME_NAME].brdbrd.net`)
   - Username
   - Password

   Example:

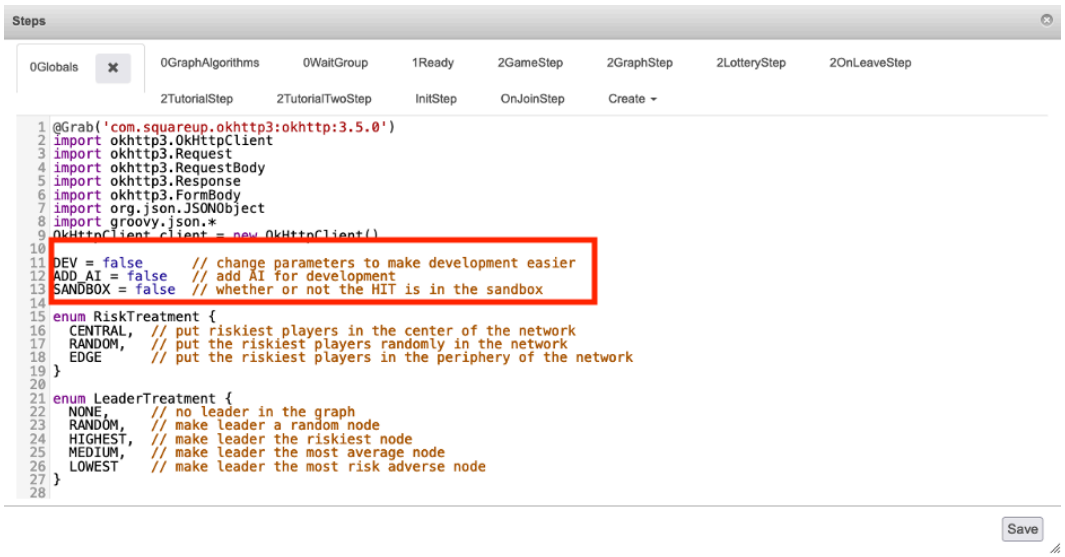
   Logging in takes you to Breadboard's console:



2. Make sure the experiment name is correct. Create a new experiment instance.
   - Note: name the instance to match the internal study name you will enter when setting up Prolific.



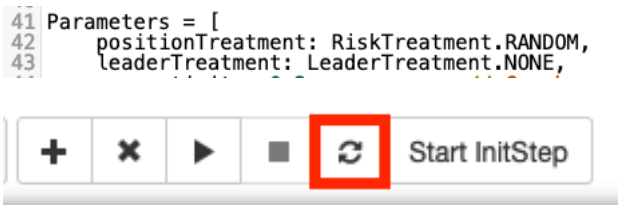
3. Click the **Steps** dialog at the bottom. In the Globals step, set the `DEV` and `SANDBOX` variables to **false**. This may not apply to all experiments.



4. This step may not apply to all experiments. Make sure the parameters in your study are correct (they are defined in the **Globals** step under **Parameters**). If your study has multiple treatment types, set

them in the Steps dialog, click **Save** at the bottom right, and reload the script engine (refresh button at the top right). You may have to refresh your page if it is loading for a while.

For example, in the revolution game, we change the risk and leader treatments here:



5. SSH into the Breadboard server through a terminal to view the experiment's logs. Ask your Breadboard administrator for the port, user and directory for your study; they look like this:

```
ssh -p <port> <study>@brdbrd.net
cd <study directory>
tail -f nohup.out
```

Your terminal should look something like this afterward:

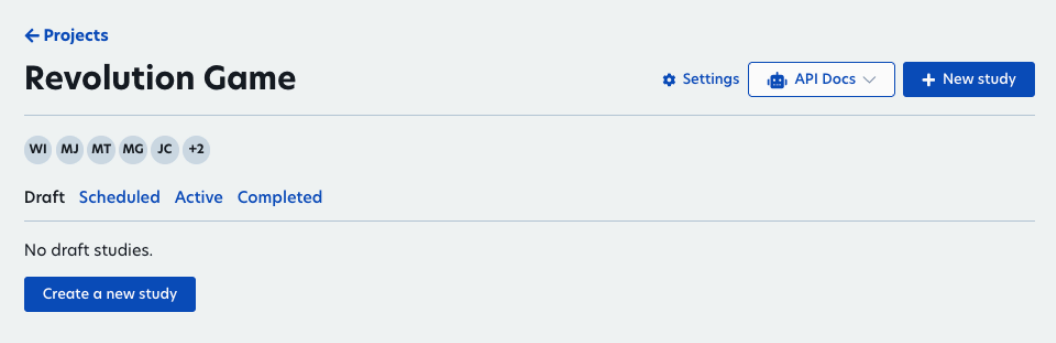


Everything is now set up on the Breadboard end. Proceed to Prolific.

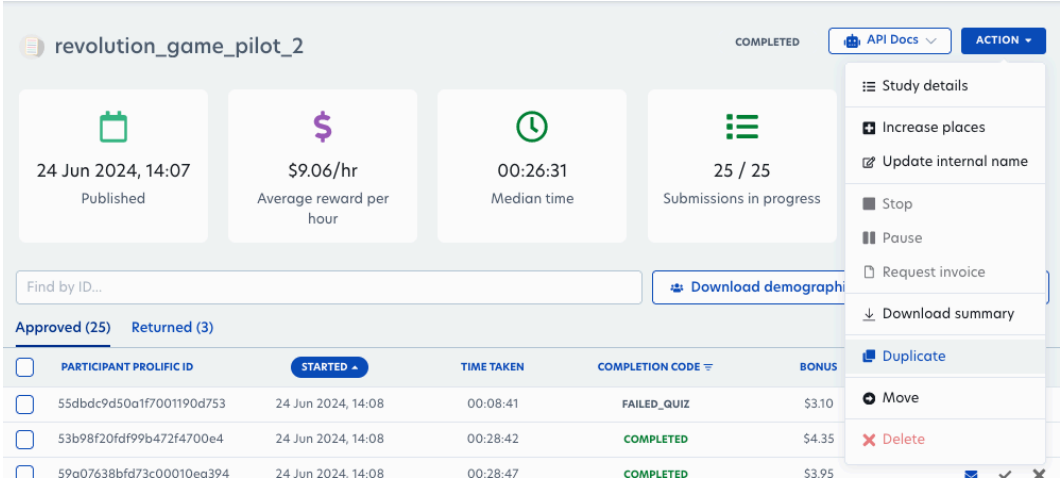
3 Setting Up Prolific

1. Select your project and create a new study. If an instance of your study already exists, you can open it and duplicate it.

   Create a new study:



   Duplicate a study:

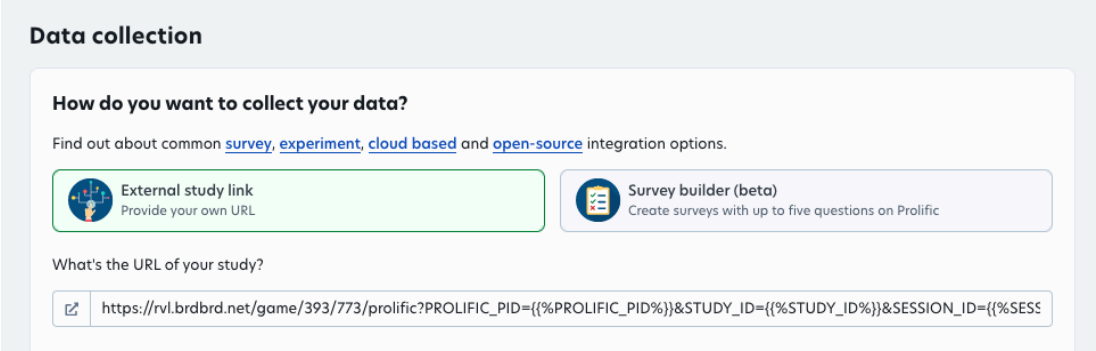


2. In the study setup, fill in the **Study details** based on what your study needs. Give the study an internal name (visible only to you) that matches the instance name you created in Breadboard.

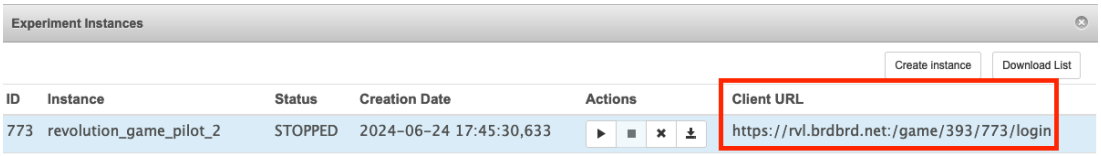
3. Under **Data collection**, answer **How do you want to collect your data?** with a study link, and enter the URL below in the field **What is the URL of your study?**, changing the bolded numbers to match the client URL of your Breadboard instance. Choose **URL Parameters** as the way to record participant IDs (select "I'll use URL parameters"), so that Prolific appends the participant's IDs.

   ```
   https://rvl.brdbrd.net/game/393/773/prolific?PROLIFIC_PID={{%PROLIFIC_PID%}}
   &STUDY_ID={{%STUDY_ID%}}&SESSION_ID={{%SESSION_ID%}}
   ```

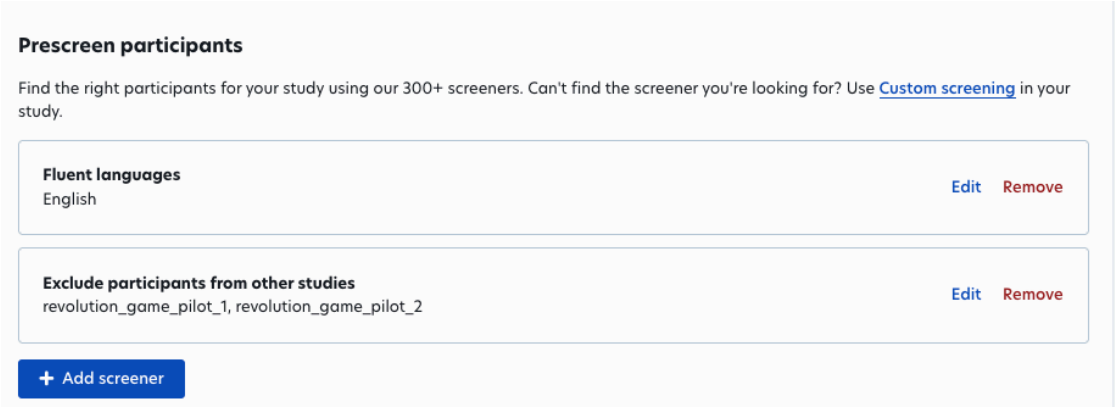
   Prolific:



Breadboard console (ignore status STOPPED; your instance should be running):



4. Under **Audience**, the **Prescreen participants** section is where you add screeners. You probably want to exclude participants from any other instances of your study, using the "Exclude participants from other studies" screener, a custom blocklist, or a participant group used as a blocklist. When you duplicate a previous study instance, its screeners carry over.



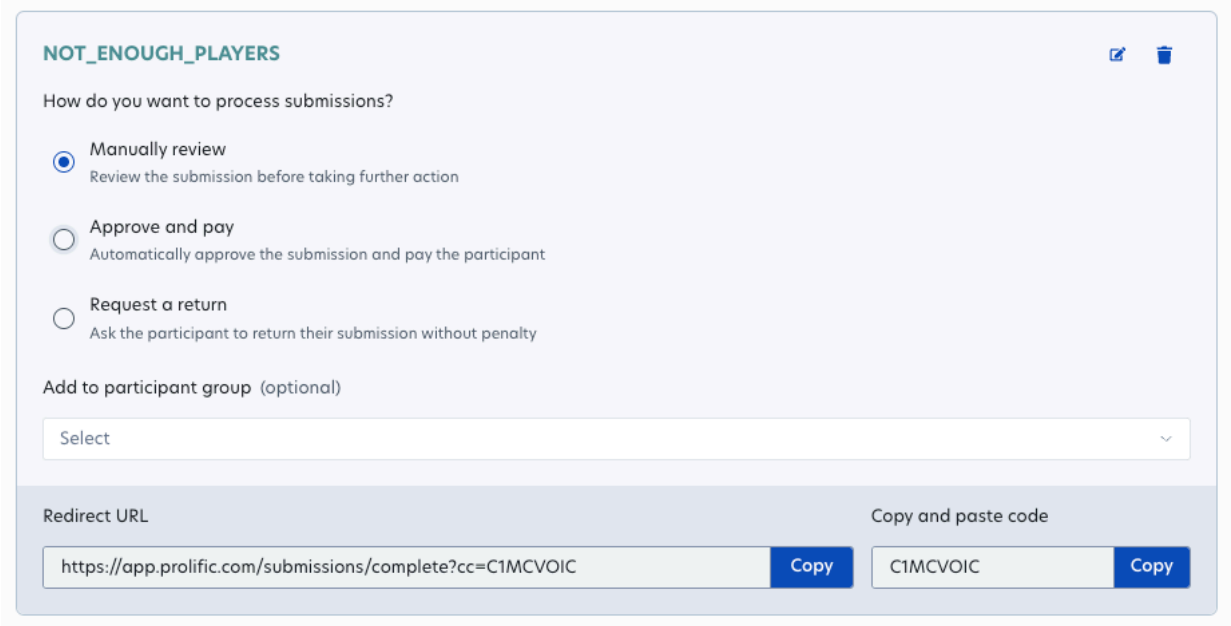
Note: Prescreeners you add are downloadable through Prolific's demographic data export (explained later). Click **Add screeners** to choose from many preset questions and to filter out participants who choose certain answers (or select them all). This is Prolific's recommended way of collecting demographic data. Prolific also recommends validating the prescreener questions at the beginning of your study. If a participant answers inconsistently with their prescreener answer, do not reject them; ask them to return their submission. To do this, word the questions in your study exactly as they are worded in the prescreener.

5. Make sure the rest of the fields match what you want for the study (see *Running Group Studies* below for the settings that matter most for multiplayer games). Use **preview** to check that everything looks right, then **publish** and confirm the final checks. Once the study is published, participants can start joining.

This concludes the Prolific setup.

3.1 Completion Paths Set Up

When participants finish your study, they return to Prolific with a completion code showing they completed it. Prolific offers three options: redirect them to the completion URL (recommended; it looks like `https://app.prolific.com/submissions/complete?cc=...`), have them copy and paste the code, or neither (submissions then show as NOCODE). A completion path can specify different conditions under which they exited the game, and each path can trigger an automatic action on Prolific (approve, or manual review, with bonuses paid separately). For example, here is a completion path from the revolution game:



Define a code for each way a participant can leave your study (finishing, failing a check, timing out, not enough players, and so on). Breadboard records which code a player exited with, so the data can be matched to participants on Prolific later, and different codes can lead to different pay. Appendix A lists the revolution game's codes and how they map to pay.

4 Pre-Launch Checklist

Go through this before you publish on Prolific.

Breadboard

- ☐ Instance name matches the internal study name you will use on Prolific.
- ☐ `DEV` and `SANDBOX` are **false** in Globals (if your study has them).
- ☐ Parameters and treatments are set, saved, and the script engine is reloaded.
- ☐ The instance is running, and the log terminal is open.
- ☐ You have tested the study with AI players or the testing dialog.

Prolific

- ☐ The study URL's experiment and instance numbers match the Breadboard client URL, and the URL parameters option is selected.
- ☐ Exclusion screeners for earlier instances of the study are in place.

- ☐ Every completion code in the Breadboard script is entered as a completion path on Prolific, with the same code.
- ☐ The time estimate and pay include expected waiting time, and any concurrency limit is set.
- ☐ You have looked at **preview**, and your wallet covers rewards plus the platform fee.
- ☐ You know when you will pause the study so that nobody joins after the game starts.

5 Running the Experiment

When running the experiment, there are three primary screens to monitor: Prolific, the Breadboard console, and the terminal logs.

5.1 Session Timeline

A typical live session runs in this order:

1. Check the pre-launch checklist, then publish the study on Prolific.
2. Watch participants arrive in the Breadboard console and the logs.
3. Once there are enough players, pause the study on Prolific so that no one joins after the game starts.
4. Let the game run, watching for idle or stuck players.
5. When the game ends, stop the study on Prolific and download the Breadboard data.
6. Approve the submissions on Prolific, then pay the bonuses.

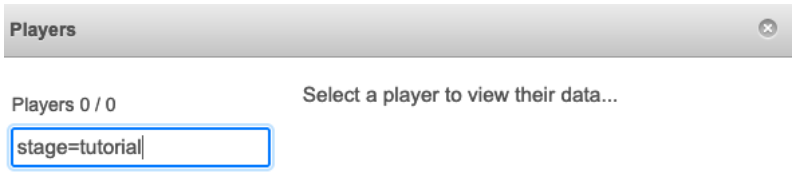
Approve promptly: Prolific approves submissions that haven't been reviewed within 21 days.

5.2 Terminal Logs

In the terminal, you can see in real time when participants join, the actions they take, and other data.

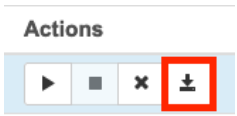
5.3 Breadboard Console

1. In the console, you can see the nodes representing each player and the choices they select each round. This is shown visually in the **Graph** dialog. To view a specific player, click the **Players** dialog at the bottom. You can filter the player data, as below, to see how many players are at a certain stage in the game (you can also filter by other parameters).



   Specific to the revolution game: to remove a player, run the `kickPlayers("Player ID")` function in the script dialog.

2. Once the experiment is done, download the data and logs by clicking the download button next to your experiment instance.



3. The experiment data is a CSV file where each line is an **event**. Events and their data are grouped together by **id**. The ids do not necessarily increase in numerical order, so to order events chronologically, sort by **event_date** instead.

id	event	event_date	data_name	data_value
131331	clientLogIn	2024-06-24T17:03:11:202Z	acceptEncoding	gzip, deflate, br, zstd
131331	clientLogIn	2024-06-24T17:03:11:202Z	acceptLanguage	en-US,en;q=0.9
131331	clientLogIn	2024-06-24T17:03:11:202Z	clientId	5d63179aa3b94d0001da4bb6
131331	clientLogIn	2024-06-24T17:03:11:202Z	connection	close
131331	clientLogIn	2024-06-24T17:03:11:202Z	host	rvl.brdbrd.net
131331	clientLogIn	2024-06-24T17:03:11:202Z	ipAddress	127.0.0.1
131331	clientLogIn	2024-06-24T17:03:11:202Z	referer	https://rvl.brdbrd.net/game/393/772/5d63179aa3b94d0001da4bb6/c
131331	clientLogIn	2024-06-24T17:03:11:202Z	requestURI	/game/393/772/5d63179aa3b94d0001da4bb6/state
131331	clientLogIn	2024-06-24T17:03:11:202Z	userAgent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHT
131332	clientLogIn	2024-06-24T17:03:13:575Z	acceptEncoding	gzip, deflate, br, zstd
131332	clientLogIn	2024-06-24T17:03:13:575Z	acceptLanguage	en-US,en;q=0.5
131332	clientLogIn	2024-06-24T17:03:13:575Z	clientId	5f4a7a00dc1bde60ab8f7ac5
131332	clientLogIn	2024-06-24T17:03:13:575Z	connection	close

5.4 Matching Players to Prolific Participants

The study URL passes `PROLIFIC_PID` (the participant ID), `STUDY_ID` and `SESSION_ID` to Breadboard. To pay bonuses or look into a problem, you need to get from a Breadboard player to a Prolific submission:

1. In the Breadboard CSV, find the player's events and the completion code they exited with.
2. Take the player ID from the CSV and look it up in Prolific's **Download demographic data** file, which lists participant IDs alongside submission IDs.
3. Use the submission ID for bonuses and approvals.

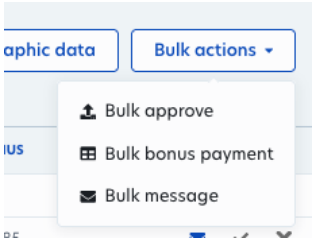
Before a real run, join as one test participant and confirm that the player ID in the Breadboard data is the Prolific participant ID. If your study records a different value, match on that value instead.

5.5 Prolific

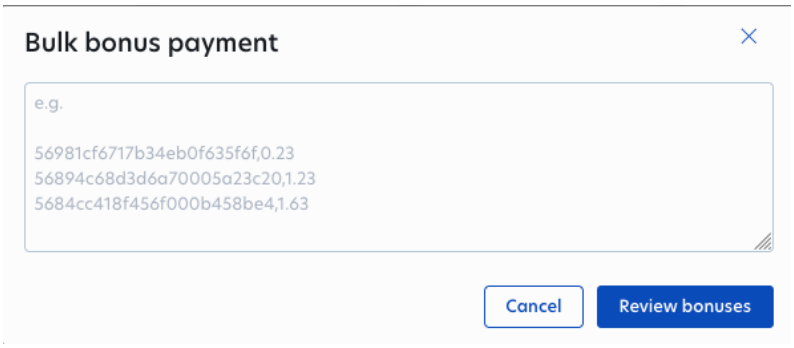
1. On Prolific, you can see the number of participants, their IDs, and other data such as time taken and completion codes. Once you have enough participants, pause the study with **Action > Pause** at the top right. Pausing does not affect participants who are already in the study. Use **Action > Stop** when the study is complete; stopping is final, though you can duplicate the study.

2. When the study is complete, go through the submissions that still need approval and approve them. To approve them all, select all and click **Approve selected** at the bottom left. Submissions that are not reviewed within 21 days are approved automatically.

3. Once participants are approved, you can pay the bonuses. Click **Bulk actions** at the top right, then **Bulk bonus payment**.



   Prolific asks for the bonus payments in the format `Submission ID,bonus amount`, one participant per line. It must be Prolific's **submission** ID, not the participant ID; using participant IDs results in failed payments. Submission IDs are at the beginning of the file from **Download demographic data** on the submissions page, next to the participant IDs, so match your player IDs to submission IDs there before pasting.



`bulk_bonus.py` (Appendix B) produces this file from the Breadboard event CSV for the revolution game. For another study, adapt it to your study's events.

4. Click **Pay Bonuses**. The update may take a moment; do not submit the same payment again unless Prolific reports that it failed. Bonuses carry Prolific's platform fee (plus VAT if applicable) and cannot be refunded, so the amount charged exceeds the total bonus the script prints.
5. Messages from participants appear in the **Messages** tab on Prolific, but only for the person who published the study, unless your workspace has enabled shared workspace messaging.

6 Running Group Studies

Prolific supplies participants but does not match them into groups; matching happens in Breadboard. These Prolific rules matter most for multiplayer games:

- **Waiting time.** Include the expected waiting time in your time estimate and pay.
- **Unmatched participants.** Anyone who waits but is never matched must receive partial pay of at least £0.10 / $0.14 per minute, and cannot be rejected. In the revolution game, the NOT_ENOUGH_PLAYERS code pays the base pay plus lottery bonus for this reason.
- **First-come, first-served.** Places fill in order of arrival, so a full group is not guaranteed.
- **Concurrency limit.** Under *Limiting simultaneous participant access* in the Data collection step, select **Limit participant access** and enter the maximum number of simultaneous participants. It is not offered for studies hosted on SurveyMonkey, Qualtrics, Google Forms, Alchemer, Typeform or Gorilla.
- **Conditions.** To assign conditions at the same time, run one Prolific study and randomize in Breadboard rather than running a separate study per condition.
- **Rejections.** Rejections are made one at a time and each needs a reason. Being slow or being screened out is not a valid reason, and Prolific sets a default rejection limit per study.
- **Pay.** Prolific sets a minimum hourly rate and recommends a higher one. You can raise a study's reward after launch but never lower it.

7 Troubleshooting

Participants cannot connect. Check that the instance is running in the console (the status column may still read STOPPED while it runs), reload the script engine, and refresh the page. Check that the experiment and instance numbers in the Prolific URL match the Breadboard client URL.

Players are stuck before the game. Check how many players are ready: the game does not start without the minimum number (15 in the revolution game), and the players who wait are dropped with the NOT_ENOUGH_PLAYERS code. For a test with `DEV` set to true, use **Start Init Step** to move players from the ready step to the game.

Players join after the game has started. Pause the Prolific study before the game starts. Late arrivals leave with the GAME_STARTED code and are not paid.

A player is idle or must be removed. In the revolution game, run `kickPlayers("Player ID")` in the script dialog. Check the exit code in the downloaded data.

Submissions show NOCODE on Prolific. The participant did not return through the completion URL or enter a code. Match them to Breadboard by participant ID and check which exit code they received.

Bonus payments fail on Prolific. The most likely cause is participant IDs in the list instead of submission IDs. Check the first column of the list, and do not resubmit a payment unless Prolific says it failed.

A participant messages you. Messages appear in the **Messages** tab for the person who published the study. If you were not that person, ask them to check.

8 Testing Experiments on Breadboard

To test your Breadboard experiment before running it on Prolific, you have two options.

8.1 Testing with AI

Use the following command in the script dialog:

```
g.addAI(a, n, [behavior])
```

Then define your own behavior for the vertices. Here is an example from the Breadboard documentation website:

```
def aiBehavior = { ai->
  // Sanity check to make sure the AI actually has choices to make
  if (ai.getProperty("choices")) {
    // A random delay is a good idea to make the AI
    // appear more human and to avoid potential race conditions
    def randomDelay = 1000 + r.nextInt(3000)
    new Timer().runAfter(randomDelay) {
      def choices = ai.getProperty("choices")
      // Choose a random choice
      def choice = choices[r.nextInt(choices.size())]
      a.choose(choice.uid)
    }
  }
}
```

```
// Add 20 AI players using the defined behavior
g.addAI(a, 20, aiBehavior)
```

8.2 Testing with Breadboard's Testing Dialog

You can also create multiple vertices and go through the experiment from the participant's perspective, using these two commands in the script dialog:

```
g.addVertices(5)
```

```
g.V.each{ it.testmode = true }
```

The first statement adds five vertices (participants) to the graph. The second puts all of them into test mode. Once you execute these commands, open the testing dialog to see the participants' screens:



The screens are small, but you can zoom in with Cmd/Ctrl and +. Alternatively, click the client URL next to the experiment instance and log in with a different participant ID (anything, as long as each is different) **on different tabs**. This lets you navigate the participant screens in a larger view than the Breadboard testing dialog.

For some games, if there is a `DEV` variable in the Globals step, you can set it to true and click the "Start Init Step" button in Breadboard to move players from the ready step into the game.

9 Data Handling

The Breadboard CSV and Prolific exports link participant IDs to behavior and to pay.

- Store them in the project's data folder, not in email or shared chats.
- Keep the participant-ID-to-submission-ID file separate from the files you analyze, and remove participant IDs from analysis copies.
- The bonus files (`bulk_bonus_payment.csv` and `metadata_bonus.csv`) contain participant or submission IDs and amounts. Delete them once the payments are confirmed.
- Prescreener answers in the Prolific export are demographic data; handle them as you would the rest of the study data.

10 Conclusion

That concludes the guide for running Breadboard experiments on Prolific. Happy experimenting!

Appendix A: Revolution Game Completion Codes

These are the codes from one revolution game study, as an example. Prolific generates a new set of codes for each study, so enter your own codes in the Breadboard script and on Prolific. The revolution game has two possible bonus payments: one from the lottery round and one from the revolution game itself. It requires at least 15 players to proceed; players dropped for lack of players leave with the NOT_ENOUGH_PLAYERS code (C1MCVOIC in the completion path shown earlier). Players are paid different bonuses depending on their code.

Completion Code Name	Prolific Code	Description	Bonus
COMPLETE	C1MTZON7	Players who completed all rounds of the experiment successfully	Base pay + both bonuses
FAILED_LOTTERY	C1HY3M7Y	Players who demonstrated irrational behavior in the lottery. Irrational behavior is defined as picking the option that has a 0% chance of payment in the first or last lottery choice, or when the number of swaps between the choices exceeds 3.	Base pay + lottery bonus
FAILED_QUIZ	CLYTL771	Players who failed the comprehension quiz at the end of the tutorial (missed more than two questions after two attempts).	Base pay + lottery bonus
PLAYER_IDLE	C8NGXRRJ	Players who were idle for too long during the game (i.e. didn't make a choice within the allotted time).	Base pay + lottery bonus
NOT_READY	CTQFLWV0	Players who did not click the ready button within the allotted time after the 20-minute timer for the tutorial ended.	Base pay + lottery bonus
DIDN'T_COMPLETE_ TUTORIAL	CO0HI6W4	Players who did not complete the tutorial within 20 minutes.	Unpaid
NOT_ENOUGH_PLAYERS	C1MCVOIC	If the number of ready players is not at least 15 by the time the game is supposed to start, all the ready players are dropped with this code.	Base pay + lottery bonus
TOO_MANY_PLAYERS	C7HOJMLQ	If the number of ready players exceeds 25 by the time the game is supposed to start, random players are dropped with this code to bring the number down to 25.	Base pay + lottery bonus
FAILED_CAPTCHA	C18LDTAB	Players who fail the captcha	Unpaid

Completion Code Name	Prolific Code	Description	Bonus
GAME_STARTED	CQ4P9QVU	Players who join when the game has already started (should not happen if the Prolific study is paused before the game starts).	Unpaid

Appendix B: Revolution Game Bonus Script

`bulk_bonus.py` turns the Breadboard event CSV (downloaded in step 2 of *Breadboard Console*) into the `submission ID,amount` list that Prolific's bulk bonus tool takes. It reads the `player`, `bonus` (in cents) and `exitType` values of each `removedPlayer` event.

```
python bulk_bonus.py EVENTS.csv --prolific-export PROLIFIC.csv
```

- `EVENTS.csv` is the Breadboard download. `PROLIFIC.csv` is Prolific's **Download demographic data** file. With it, the script swaps each player's participant ID for the matching submission ID, and writes nothing if any paid player has no match. Without it, the script warns that the output uses Breadboard player IDs.
- It writes `bulk_bonus_payment.csv` (paste into Prolific) and `metadata_bonus.csv` (IDs, amounts and exit types, for reference).
- It prints the number of players, the number of payments and the total bonus. Compare these with Prolific before you pay.
- A player removed more than once is paid once, for the earliest removal.

```
"""
Format bulk bonus payments for Prolific from a Breadboard event export.

Usage:
    python bulk_bonus.py EVENTS.csv [--prolific-export PROLIFIC.csv]

EVENTS.csv is the CSV downloaded from the Breadboard console. Each
"removedPlayer" event carries the player, bonus (in cents) and exitType.

Prolific's bulk bonus tool needs submission IDs. If the Breadboard "player"
value is the Prolific participant ID, pass the Prolific "Download demographic
data" file with --prolific-export and the script swaps in submission IDs. It
refuses to write the payment file if any payee has no match.

Writes bulk_bonus_payment.csv ("id,amount", paste into Prolific) and
metadata_bonus.csv (id, amount, exit type, for reference), then prints a
summary to check against what Prolific shows before you pay.
"""
import argparse
import sys

import pandas as pd
```

```
EVENT_COLUMNS = {"id", "event", "event_date", "data_name", "data_value"}


def find_column(df, candidates, label):
    """Find a column by name, ignoring case, spaces and underscores."""
    norm = {c.lower().replace(" ", "").replace("_", ""): c for c in df.columns}
    for cand in candidates:
        if cand in norm:
            return norm[cand]
    sys.exit(f"Could not find the {label} column in the Prolific export. "
             f"Columns present: {list(df.columns)}")


def removed_players(events):
    """One row per removedPlayer event: player, bonus (dollars), exit type."""
    rows = []
    removed = events[events["event"] == "removedPlayer"]
    for _, group in removed.groupby("id"):
        values = {}
        for _, row in group.iterrows():
            values[row["data_name"]] = row["data_value"]
        if "player" in values and "bonus" in values:
            rows.append({
                "player": str(values["player"]),
                "bonus": round(float(values["bonus"]) / 100, 2),
                "exit_type": values.get("exitType"),
                "event_date": group["event_date"].min(),
            })
    return pd.DataFrame(rows, columns=["player", "bonus", "exit_type", "event_date"])


def main():
    parser = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument("events", help="Breadboard event CSV")
    parser.add_argument("--prolific-export", help="Prolific 'Download demographic
data' CSV")
    args = parser.parse_args()

    events = pd.read_csv(args.events, dtype=str)
    missing = EVENT_COLUMNS - set(events.columns)
    if missing:
        sys.exit(f"{args.events} is missing columns {sorted(missing)}; "
                 "is this the Breadboard event export?")

    players = removed_players(events)
    if players.empty:
        sys.exit("No removedPlayer events with a player and bonus were found.")

    # A player can be removed more than once; pay only the first removal.
```

```
    players = players.sort_values("event_date")
    duplicates = players["player"].duplicated().sum()
    players = players.drop_duplicates("player")

    if args.prolific_export:
        export = pd.read_csv(args.prolific_export, dtype=str)
        pid_col = find_column(export, ["participantid"], "participant ID")
        sid_col = find_column(export, ["submissionid", "sessionid"], "submission ID")
        lookup = dict(zip(export[pid_col], export[sid_col]))
        players["pay_id"] = players["player"].map(lookup)
        unmatched = players[players["pay_id"].isna() & (players["bonus"] != 0)]
        if not unmatched.empty:
            sys.exit("No Prolific submission found for: "
                     + ", ".join(unmatched["player"]) + "\nNothing written.")
    else:
        print("WARNING: no --prolific-export given; the output uses Breadboard "
              "player IDs, which Prolific's bulk bonus tool rejects unless they "
              "are submission IDs.")
        players["pay_id"] = players["player"]

    paid = players[players["bonus"] != 0]
    with open("bulk_bonus_payment.csv", "w") as out:
        for _, row in paid.iterrows():
            out.write(f"{row['pay_id']},{row['bonus']:.2f}\n")
    with open("metadata_bonus.csv", "w") as out:
        for _, row in players.iterrows():
            out.write(f"{row['pay_id']},{row['bonus']:.2f},{row['exit_type']}\n")

    print(f"Players removed: {len(players)} ({duplicates} repeat removals ignored)")
    print(f"Payments written: {len(paid)} ({len(players) - len(paid)} with no
bonus)")
    print(f"Total bonus: {paid['bonus'].sum():.2f}")


if __name__ == "__main__":
    main()
```